

Mathematical Foundations of the Imagized Language Model (ILM)

Lachlan Chen AgInTiFlow
AgInTi Lab, LazyingArt LLC

May 31, 2026

Abstract

The Imagized Language Model (ILM) learns semantic representations directly from glyph images of words or characters, bypassing traditional tokenization. Using Alpaca-style question-answer (QA) data, ILM jointly trains: (i) a glyph encoder mapping 128×128 images to a latent vector, (ii) a multi-channel product codebook that yields discrete, memory-like indices per token, and (iii) contrastive objectives linking glyphs, words, and sentence semantics. This note formalizes the data pipeline, model components, and training losses, and shows correspondence with the implemented training code.

1 Overview

Given a corpus of QA pairs, ILM optimizes three aligned representations:

- Glyph encoder $f_g : \mathbb{R}^{3 \times H \times W} \rightarrow \mathbb{R}^d$ (with $H = W = 128$).
- Product codebook embedding $f_c : t \mapsto \mathbf{e}_t \in \mathbb{R}^d$ with multi-channel discrete indices.
- Contrastive and structural objectives: (glyph $\leftrightarrow$ code) InfoNCE, (question $\leftrightarrow$ answer) InfoNCE, and Gram alignment.

Supervision arises from self-alignment: glyph $\leftrightarrow$ code embeddings and question $\leftrightarrow$ answer semantics.

2 Data and Tokenization

For a QA pair (Q_i, A_i) , language-dependent tokenization yields

$$T_Q = [t_1^Q, \dots, t_m^Q], \quad T_A = [t_1^A, \dots, t_n^A].$$

English uses regex wordpiece tokenization; Chinese uses per-character segmentation over CJK code points. Each token t is rendered as a glyph image $x_t \in [0, 1]^{3 \times 128 \times 128}$, via a language-specific renderer (dynamic tiling for EN; centered font glyph for ZH).

3 Glyph Encoder

A lightweight CNN f_g maps a glyph to a d -dimensional embedding, followed by L2 normalization:

$$\mathbf{g}_t = \frac{f_g(x_t)}{\|f_g(x_t)\|_2} \in \mathbb{R}^d.$$

Adaptive pooling provides invariance to the exact tiling layout used for shorter words.

4 Product Codebook Embedding

We define a multi-channel codebook with C channels and K codes per channel. For channel c , the code vectors are

$$E_c = [\mathbf{e}_{c,1}, \dots, \mathbf{e}_{c,K}] \in \mathbb{R}^{K \times (d/C)}, \quad c = 1, \dots, C.$$

Each token j has learnable assignment logits $A_j \in \mathbb{R}^{C \times K}$. With temperature τ , channel-wise selection probabilities are

$$p_{j,c,k} = \frac{\exp(A_{j,c,k}/\tau)}{\sum_{k'} \exp(A_{j,c,k'}/\tau)}.$$

The token embedding is the concatenation of channel aggregates:

$$\mathbf{e}_j = \text{concat}_{c=1}^C \sum_{k=1}^K p_{j,c,k} \mathbf{e}_{c,k} \in \mathbb{R}^d, \quad \hat{\mathbf{e}}_j = \frac{\mathbf{e}_j}{\|\mathbf{e}_j\|_2}.$$

As $\tau \rightarrow 0$, the selection becomes hard: $c_j^*(c) = \arg \max_k A_{j,c,k}$. The discrete tuple $\text{addr}(j) = (c_j^*(1), \dots, c_j^*(C))$ serves as a compact, memory-like index for token j .

5 Sentence Embeddings

Sentence embeddings are averages of token embeddings:

$$\mathbf{s}_Q = \frac{1}{|T_Q|} \sum_{t \in T_Q} \mathbf{e}_t, \quad \mathbf{s}_A = \frac{1}{|T_A|} \sum_{t \in T_A} \mathbf{e}_t,$$

with normalized forms $\hat{\mathbf{s}}_Q = \mathbf{s}_Q / \|\mathbf{s}_Q\|_2$ and $\hat{\mathbf{s}}_A = \mathbf{s}_A / \|\mathbf{s}_A\|_2$.

6 Objective Functions

6.1 Glyph–Code InfoNCE

For U unique tokens in a mini-batch, with cosine similarities $S_{uv} = \hat{\mathbf{g}}_u^\top \hat{\mathbf{e}}_v$, the InfoNCE objective is

$$\mathcal{L}_{\text{img}} = -\frac{1}{U} \sum_{u=1}^U \log \frac{\exp(S_{uu}/\tau_g)}{\sum_{v=1}^U \exp(S_{uv}/\tau_g)}.$$

6.2 QA InfoNCE (Semantic Alignment)

For B QA pairs, define similarities $M_{ij} = \hat{\mathbf{s}}_{Q_i}^\top \hat{\mathbf{s}}_{A_j}$. The QA InfoNCE loss is

$$\mathcal{L}_{\text{qa}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(M_{ii}/\tau_s)}{\sum_{j=1}^B \exp(M_{ij}/\tau_s)}.$$

6.3 Gram Matrix Structural Loss

Using token-level normalized embeddings inside sentences, form $Z_Q = [\hat{\mathbf{e}}_{t_1^Q}, \dots, \hat{\mathbf{e}}_{t_m^Q}]^\top$, $Z_A = [\hat{\mathbf{e}}_{t_1^A}, \dots, \hat{\mathbf{e}}_{t_n^A}]^\top$. The Gram matrices are $G_Q = Z_Q Z_Q^\top$ and $G_A = Z_A Z_A^\top$. With $m_0 = \min(m, n)$,

$$\mathcal{L}_{\text{gram}} = \frac{1}{B} \sum_{i=1}^B \frac{1}{m_{0,i}^2} \|G_{Q_i}^{(m_{0,i})} - G_{A_i}^{(m_{0,i})}\|_F^2.$$

6.4 Code Usage Entropy Regularization

Let $P = \text{softmax}(A, \text{dim} = K) \in \mathbb{R}^{V \times C \times K}$ be assignment probabilities over the vocabulary V . The average entropy promotes diverse code usage:

$$H = -\frac{1}{VC} \sum_{j=1}^V \sum_{c=1}^C \sum_{k=1}^K P_{j,c,k} \log P_{j,c,k}, \quad \mathcal{L}_{\text{reg}} = -\lambda_u H.$$

6.5 Total Loss

Combining the terms (with typical hyperparameters $\tau_g = \tau_s = 0.07$, $\lambda_g = 0.1$, $\lambda_u = 0.01$):

$$\mathcal{L} = \mathcal{L}_{\text{img}} + \mathcal{L}_{\text{qa}} + \lambda_g \mathcal{L}_{\text{gram}} + \mathcal{L}_{\text{reg}}.$$

7 Gradient Behavior and Learning Dynamics

Term	Gradient target	Semantic effect
$\mathcal{L}_{\text{img}}$	glyph encoder f_g , codebooks	visual–symbolic consistency
$\mathcal{L}_{\text{qa}}$	sentence means $\mathbf{s}_Q, \mathbf{s}_A$	semantic grouping/alignment
$\mathcal{L}_{\text{gram}}$	token geometry within Q/A	structural/grammatical consistency
$\mathcal{L}_{\text{reg}}$	assignment logits A	anti-collapse, code diversity

8 Consistency with Implementation

The training step mirrors the formulas:

- Stage 1: compute normalized glyph and code embeddings for unique tokens; apply InfoNCE on the $U \times U$ cosine matrix.
- Stage 2: compute sentence means for QA pairs (normalized); apply QA InfoNCE on the $B \times B$ cosine matrix.
- Gram alignment: build normalized token matrices for each Q and A; minimize Frobenius distance between Gram matrices (size matched by m_0).
- Regularization: add negative mean assignment entropy to encourage broad code usage per channel.

9 Semantic Emergence and Discrete Memory Indices

The product codebook induces a factorized latent space: each channel captures a complementary aspect (e.g., lexical category, domain, morphological pattern), and the concatenation composes them. InfoNCE terms pull together tokens that serve similar roles across QA, while Gram alignment encourages relational structure. The learned hard indices $\text{addr}(j)$ then serve as a compact memory address per token, suitable for table lookups or hierarchical indexing.

10 Diffusion Extension (Optional)

To extend from static embeddings to generative modeling over glyph grids, one can train a UNet to invert a corruption process on glyph frames. With standard variance-preserving diffusion,

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I),$$

the training objective is

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \boldsymbol{\epsilon}} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2,$$

applied to glyph feature maps or latent embeddings. This yields coherent in-filling and iterative refinement of text-like images.

11 Conclusion

ILM jointly optimizes visual, structural, and semantic coherence by connecting glyph encodings with a discretized, factorized codebook and QA-driven alignment. The resulting representation learns semantics without external lexicons and produces discrete memory indices for efficient downstream use. The objectives above match the implemented pipeline.